Anaí Lael Barrera Magaña

Experimentación reproducible de un modelo de Churn con MLflow

Enlace

Objetivo y procedimiento

Se migró el modelo base de abandono de clientes del notebook exploratorio a un proyecto Python ejecutable y parametrizado, con una organización inspirada en CCDS. El entrenamiento se concentra en churn/modeling/train.py; la limpieza, el preprocesamiento y los gráficos se separaron en módulos para facilitar su mantenimiento y reproducción.

Se utilizó el archivo Telco Customer Churn:

- La variable objetivo es Churn (Sí/No).
- TotalCharges se transforma a numérico y los valores vacíos se sustituyen por cero.
- El conjunto se divide en 80 % para entrenamiento y 20 % para prueba, conservando la proporción de clases (stratify) y utilizando una semilla de 42.
- La codificación y estandarización se ajustan solo con datos de entrenamiento mediante un Pipeline.

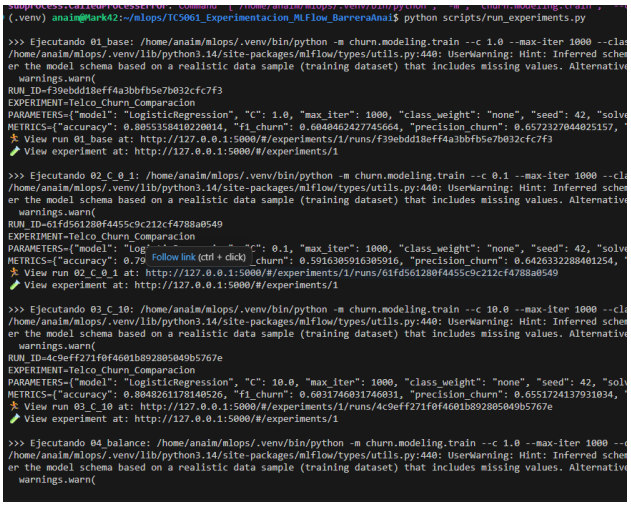
Parámetros y registro de experimentos

El script expone C (regularización), max_iter (límite de iteraciones), class_weight (ponderación de clases) y seed (aleatoriedad), todos con valores predeterminados. Cada ejecución registra sus parámetros, métricas (accuracy, precision, recall, F1 y ROC AUC), matriz de confusión, archivo JSON de métricas y modelo entrenado en el experimento Telco_Churn_Comparacion.

Comandos de ejecución y repositorio

Corrida predeterminada: python -m churn.modeling.train

Cinco configuraciones: python scripts/run_experiments.py



Anaí Lael Barrera Magaña

Análisis de los resultados

La comparación de las cinco corridas registradas en MLflow permitió identificar diferencias en el desempeño del modelo de Regresión Logística al modificar sus hiperparámetros. La configuración 04_balance obtuvo el mayor F1-score (0.614) y recall (0.783) para la clase de abandono, superando al modelo base, cuyo recall fue de 0.559.

El hiperparámetro que produjo la variación más importante fue class_weight, ya que permitió dar mayor importancia a la clase minoritaria durante el entrenamiento. Aunque esta configuración redujo la exactitud general y la precisión, mejoró considerablemente la capacidad de identificar clientes que abandonan el servicio.

Por otro lado, las modificaciones del parámetro C y del número máximo de iteraciones generaron diferencias pequeñas en las métricas. Esto demuestra que una configuración con mayor exactitud no necesariamente es la más adecuada para identificar clientes en riesgo.

Como limitación, los resultados se obtuvieron utilizando una sola partición de los datos. En futuras pruebas sería conveniente incorporar validación cruzada, analizar el costo de los falsos positivos y ampliar la búsqueda de hiperparámetros para evaluar la estabilidad y utilidad del modelo.

Reproducibilidad

Se mantuvo la misma semilla (42), el mismo archivo de datos, la misma partición estratificada y el mismo algoritmo (Regresión Logística con solver liblinear) en todas las corridas.

Las versiones de Python y de las bibliotecas se documentan en python_env.yaml y requirements.txt. El paquete incluye una prueba automatizada que comprueba que dos entrenamientos con igual configuración y semilla producen la misma métrica sobre los mismos datos.

Limitaciones y conclusiones

- La comparación utiliza una única división entrenamiento/prueba, por lo que no sustituye una validación cruzada.
- A partir de la comparación de las cinco corridas registradas en MLflow, se identificó que la configuración [indicar configuración ganadora] obtuvo el mejor desempeño según la métrica F1-score, con un valor de [resultado]. Al comparar las configuraciones evaluadas, se observó que el hiperparámetro [indicar hiperparámetro] produjo la variación más relevante en [métrica]. Este resultado permite reconocer que la selección de hiperparámetros debe estar relacionada con el objetivo del problema y sustentarse en las métricas registradas durante la experimentación.
- En datos desbalanceados, class_weight puede mejorar el recall y afectar la precisión. max_iter puede no influir si el optimizador alcanza convergencia antes del límite.

Anaí Lael Barrera Magaña

Por ello, el resultado seleccionado representa la mejor configuración observada dentro de estas pruebas, no necesariamente la óptima general.

La realización de esta actividad permite comprender la importancia de pasar de un desarrollo exploratorio en Jupyter Notebook a un proceso de entrenamiento estructurado y reproducible. Al organizar el código en funciones y separar las etapas de carga de datos, preprocesamiento, entrenamiento y evaluación, se facilita el mantenimiento del proyecto y la ejecución de nuevos experimentos sin modificar directamente el código fuente.

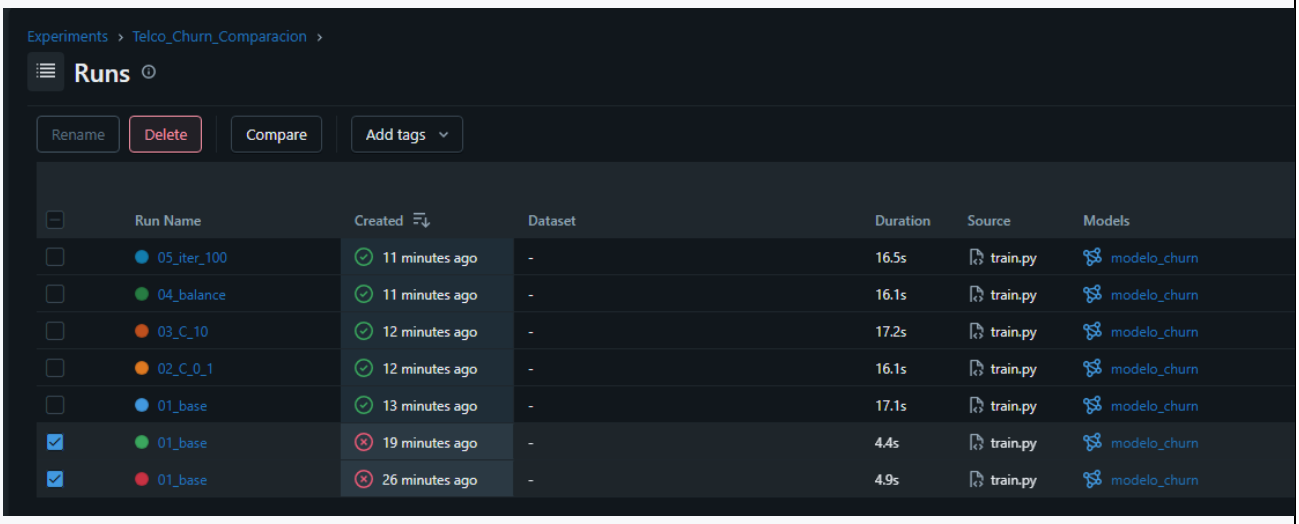
La incorporación de MLflow representa una mejora importante en la administración del ciclo de vida del modelo. El registro de hiperparámetros, métricas, artefactos y versiones del modelo permite mantener un historial de los experimentos realizados. Esto facilita identificar las configuraciones utilizadas y comparar sus resultados, evitando depender de anotaciones manuales o ejecuciones aisladas.

7. Evidencias

1. Tabla de corridas en MLflow

Mostrar al menos cinco corridas del experimento Telco_Churn_Comparacion, sus parámetros y sus métricas (F1 y recall).

CAPTURA A — TABLA CON CINCO CORRIDAS



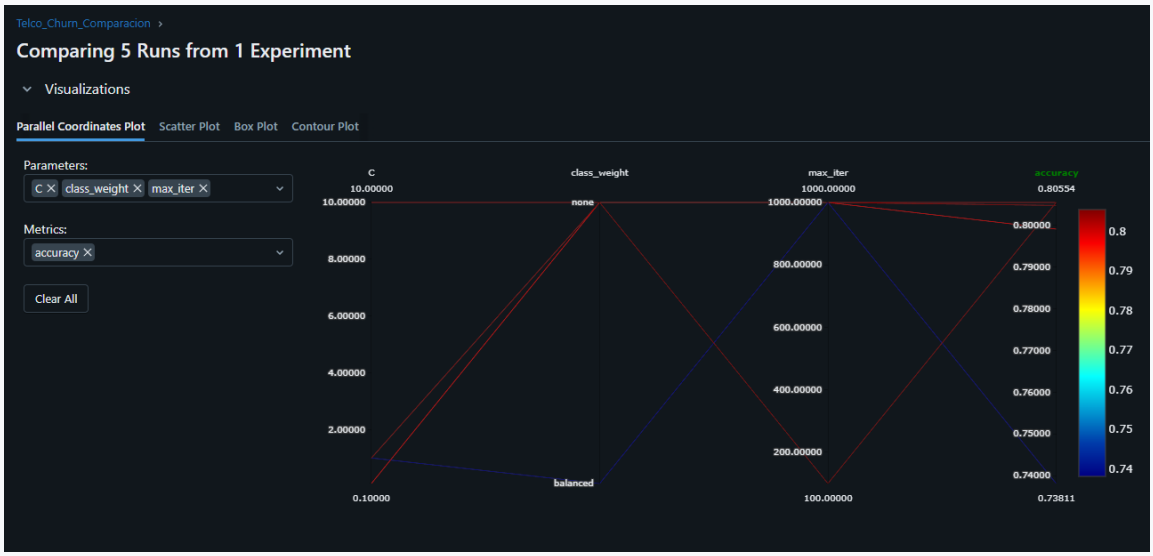
Salieron 2 con error por el cambio del nombre del dataset limpio

2. Comparación entre corridas

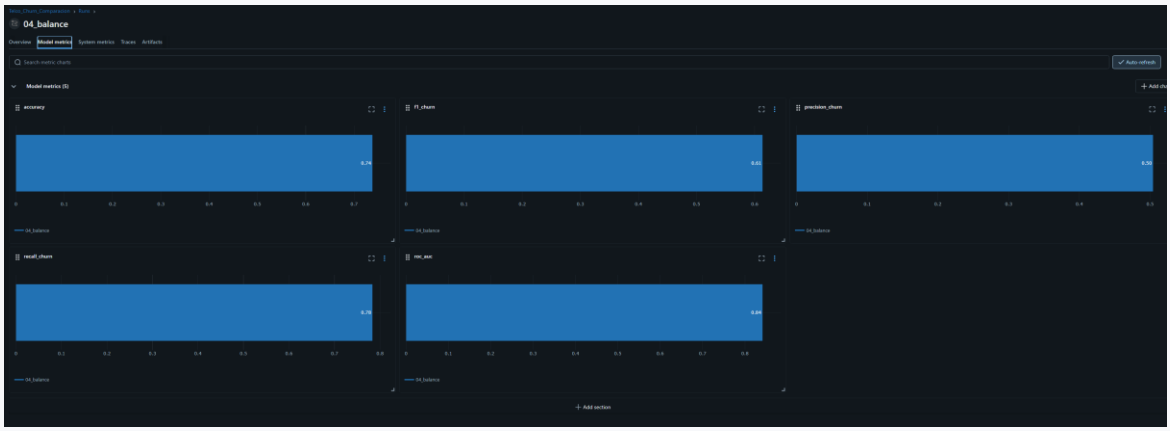
Tabla o gráfico que permite comparar sus métricas.

Anaí Lael Barrera Magaña

CAPTURA B — COMPARACIÓN EN MLFLOW



CAPTURA C



Detalle 04_balance

Anaí Lael Barrera Magaña

3. Detalles de comparación de corridas

Comparación de corridas

CAPTURAS D — DETALLE: PARÁMETROS Y MÉTRICAS

Parameters

Show diff only

C	1.0	0.1	10.0	1.0	1.0
class_weight	none	none	none	balanced	none
dataset	WA_Fn-UseC_-Telco-Customer-Churn.csv	WA_Fn-UseC_-Telco-Customer-Churn.csv	WA_Fn-UseC_-Telco-Customer-Churn.csv	WA_Fn-UseC_-Telco-Customer-Churn.csv	WA_Fn-UseC_-Telco-Customer-Churn.csv
max_iter	1000	1000	1000	1000	100
model	LogisticRegression	LogisticRegression	LogisticRegression	LogisticRegression	LogisticRegression
seed	42	42	42	42	42
solver	liblinear	liblinear	liblinear	liblinear	liblinear
test_samples	1409	1409	1409	1409	1409
test_size	0.2	0.2	0.2	0.2	0.2
train_samples	5634	5634	5634	5634	5634

Metrics

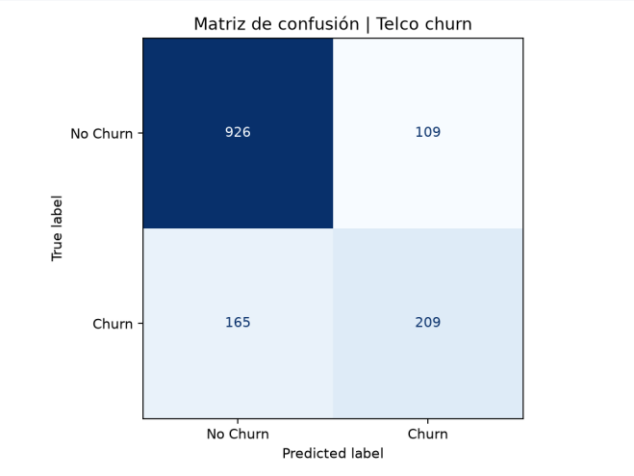
Show diff only

accuracy	0.806	0.799	0.805	0.738	0.806
f1_churn	0.604	0.592	0.603	0.614	0.604
precision_churn	0.657	0.643	0.655	0.504	0.657
recall_churn	0.559	0.548	0.559	0.783	0.559
roc_auc	0.842	0.841	0.841	0.842	0.842

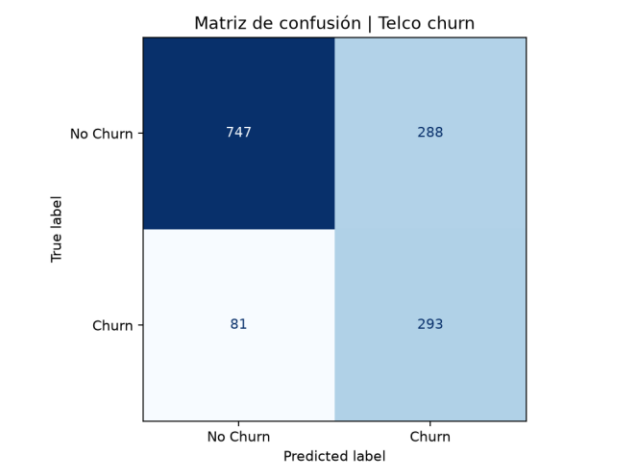
Anaí Lael Barrera Magaña

CAPTURAS F — ARTIFACTS: MATRIZ DE CONFUSIÓN

mlflow-artifacts:/1/f39ebdd18eff4a3bbfb5e7b032cfc7f3/artifacts/evaluacion/matriz_confusion.png



01_BASE



mlflow-artifacts:/1/28ee229de11b469a94248326d8960848/artifacts/evaluacion/matriz_confusion.png

04_BALANCE